CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 8: Binary Tree Traversals and Binary Search Tree Insertion

Practical Aim: To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree.

Problem 1:

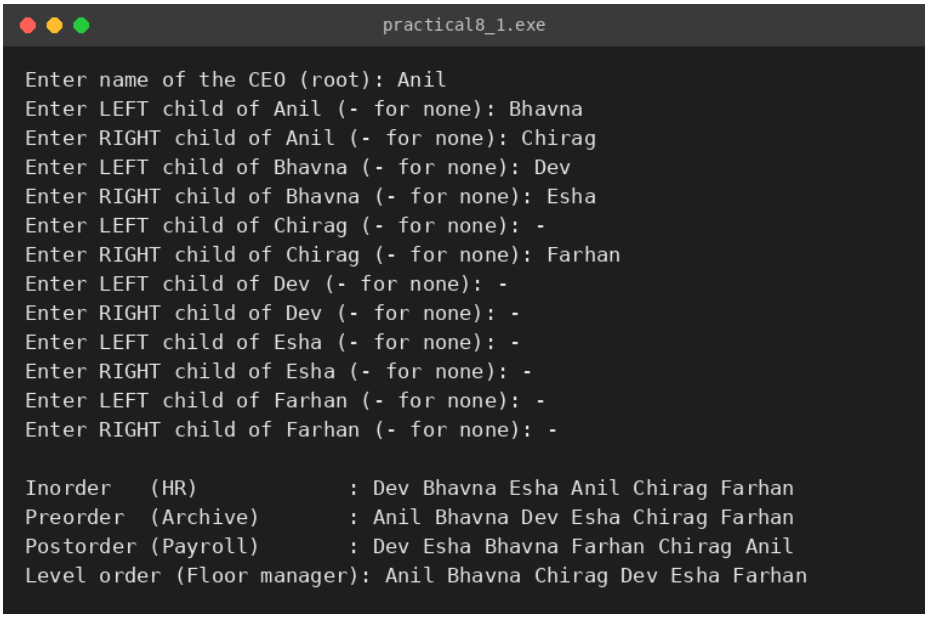
A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each.

Approach: The tree is read level by level from the user (a '-' means no child) using a queue, and stored as nodes with left and right pointers. Inorder (Left, Root, Right), preorder (Root, Left, Right) and postorder (Left, Right, Root) are written recursively: the three functions are identical except for the position of the line that prints the current node. Level order cannot be done with the same recursion, because recursion goes deep into one subtree before visiting the others, whereas level order must move across all nodes of one level before going to the next. It therefore uses a queue: pop a node, print it, and push its children, which are served after the nodes already waiting.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-8/practical8_1.cpp

Output :



Problem 2:

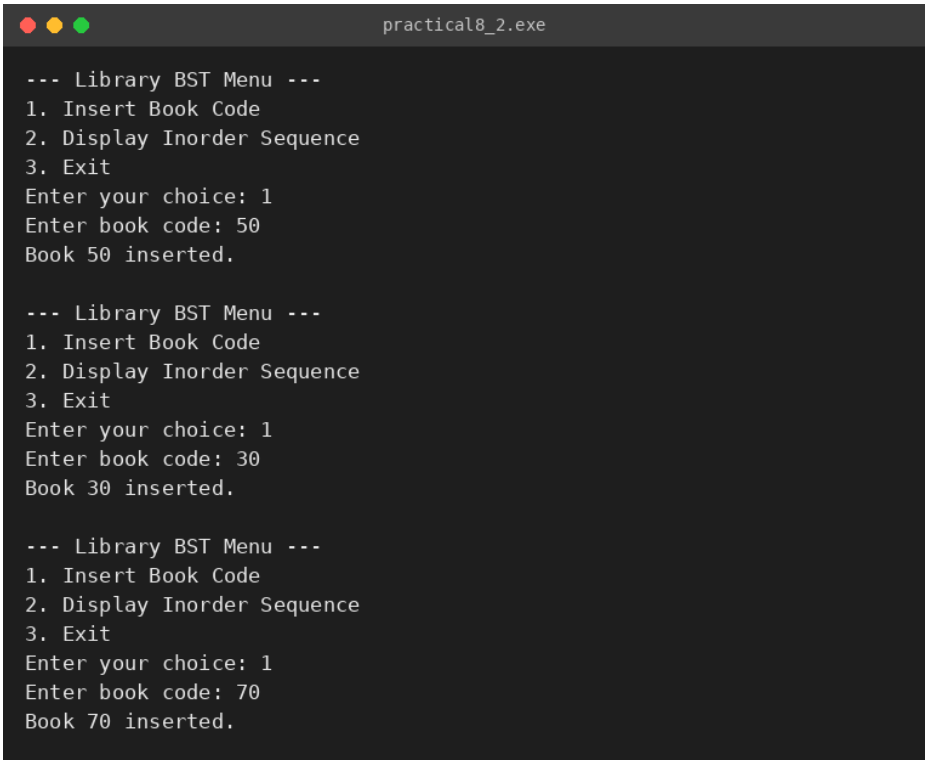
A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete.

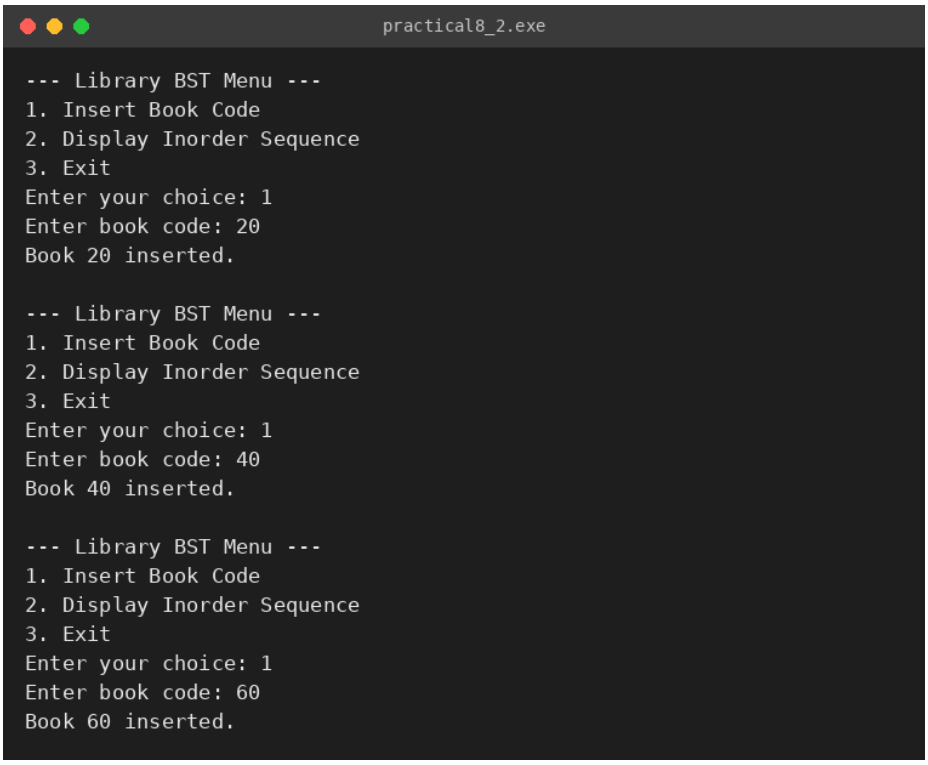
Approach: Each new code is compared with the current node starting from the root: a smaller code goes to the left subtree and a larger code goes to the right subtree. This is repeated recursively until an empty (null) position is found, and a new node is created there, which creates the new link from its parent. Duplicate codes are rejected because codes are unique. After all insertions, the inorder traversal is printed; for a correctly built BST the inorder sequence is strictly increasing, and the program also checks this automatically (Sorted check: PASS/FAIL).

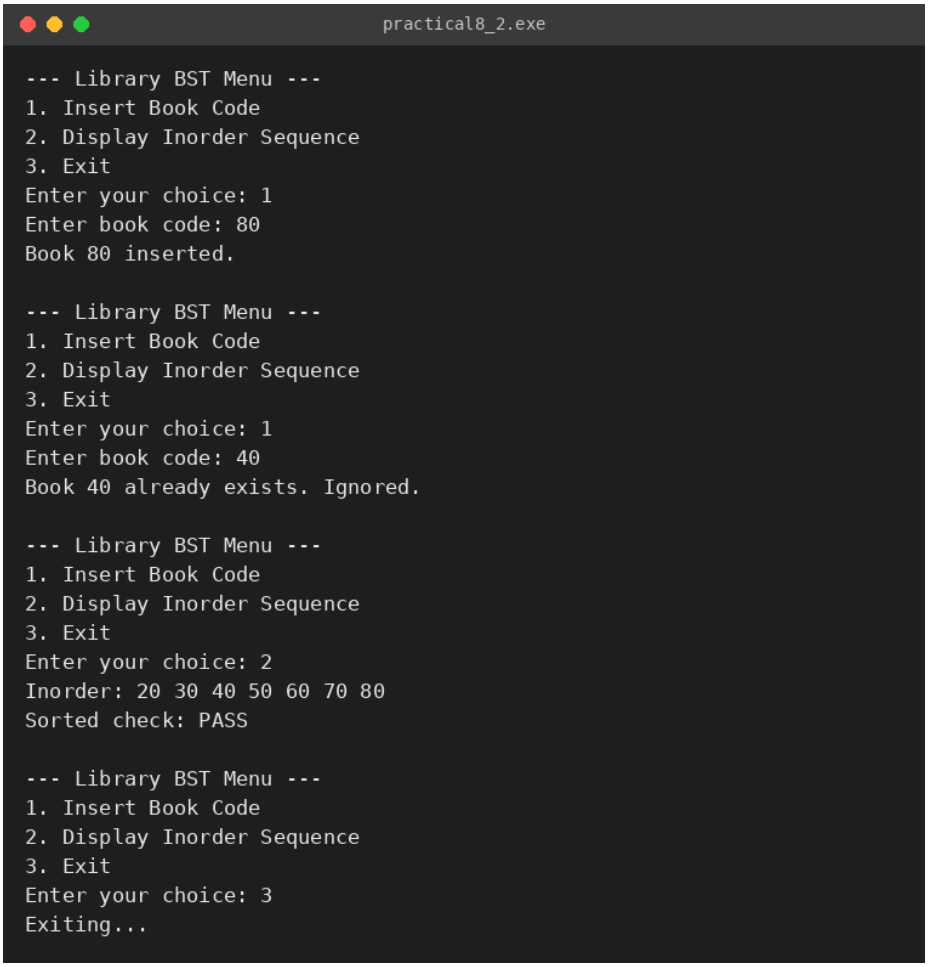
For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-8/practical8_2.cpp

Output :


practical8_2.exe
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 50
Book 50 inserted.
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 30
Book 30 inserted.
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 70
Book 70 inserted.


practical8_2.exe
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 20
Book 20 inserted.
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 40
Book 40 inserted.
--- Library BST Menu ---
1. Insert Book Code
2. Display Inorder Sequence
3. Exit
Enter your choice: 1
Enter book code: 60
Book 60 inserted.



KEY QUESTIONS:

Problem 1: Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

Problem 2: The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both? How would you isolate which one?

ANSWERS:

ANS-1: No, a preorder sequence alone is not enough to reconstruct a general binary tree. Preorder only tells us that the first value is the root, but it does not tell us how the remaining values are split between the left and right subtrees. Different tree shapes can give the same

preorder output; for example, a root A with only a left child B and a root A with only a right child B both give 'A B'.

Additional information is needed: either the inorder traversal as well (preorder + inorder uniquely identifies a tree with distinct values), or null markers for empty children in the preorder output (serialization), or the knowledge that the tree is a BST, in which case the left/right split can be found from the values themselves.

ANS-2: A non-sorted inorder output means that at least one of the two parts is wrong, and it can be either of them. If the insertion logic is wrong (for example a node is placed on the wrong side because of a wrong comparison), the tree itself no longer satisfies the BST property, so even a correct inorder traversal prints an unsorted sequence. If the traversal logic is wrong (for example the visit is printed at the wrong position), a correct BST would still be printed in the wrong order.

To isolate the problem, first test the traversal on a small tree that is built by hand and known to be correct (e.g. root 2 with children 1 and 3 must print 1 2 3). If that works, the traversal is fine and the insertion is faulty; then print the tree after each insertion, or check at every node that left < node < right, to find the first insertion that breaks the property.